Student name : Mohammad Hanbali
register number : 12219922
instructor : Bashar Tahayneh

1. Project Overview

Object Match Studio is a graphical desktop application built with Python (Tkinter, OpenCV). It demonstrates classical Digital Image Processing (DIP) techniques for object localization – finding a small query object inside a larger scene image.

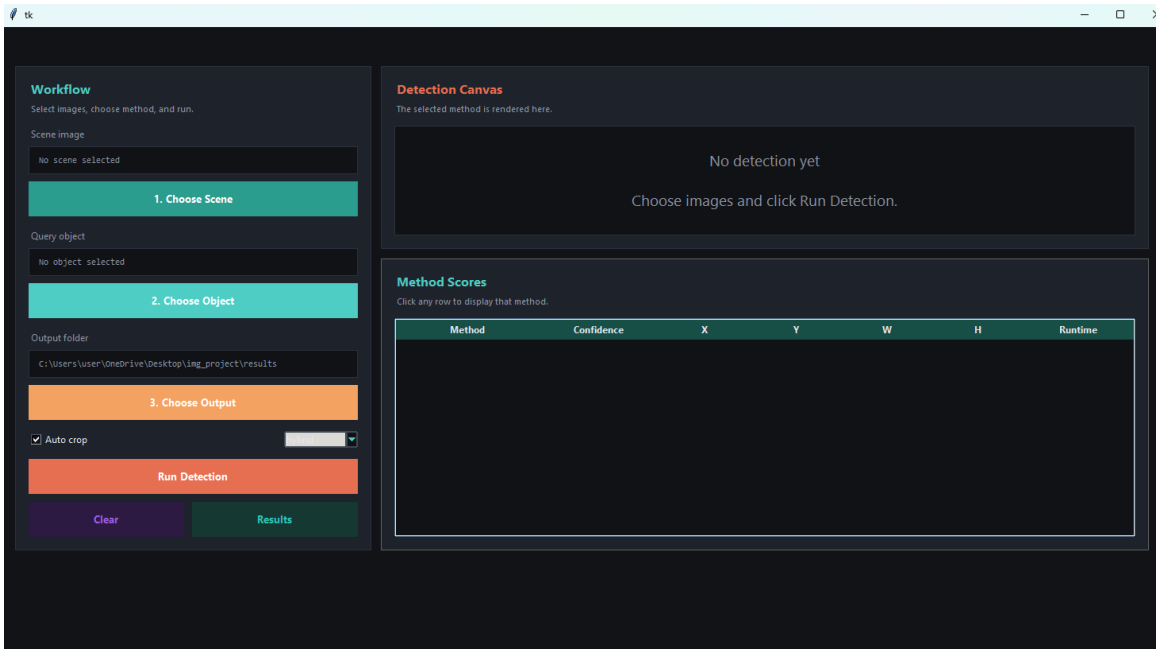
The user provides:

- A scene image (e.g., a cluttered photo)
- An object image (e.g., a cropped picture of the item to find)

The application runs four different detection methods:

1. Template Matching (correlation-based)
2. Edge Detection (using Canny + contour matching)
3. Color Histogram (back-projection + thresholding)
4. Hybrid Fusion (combines the above for better accuracy)

Results are displayed visually (bounding box drawn on the scene) and in a table with confidence scores, coordinates, and runtime. The user can switch between methods, save outputs, and export JSON summaries.



2. GUI Layout (Two-Column Design)

After removing the inspector panel, the interface consists of:

- Header – Title, subheading, stage indicator, and a tagline.
- Workflow Panel (left column) – All controls for selecting images, choosing method, running detection, clearing, and opening output folder.
- Preview Panel (right column) – Top area shows the detection result (scene with bounding box), bottom area shows a table of method scores.

Below is a breakdown of each area.

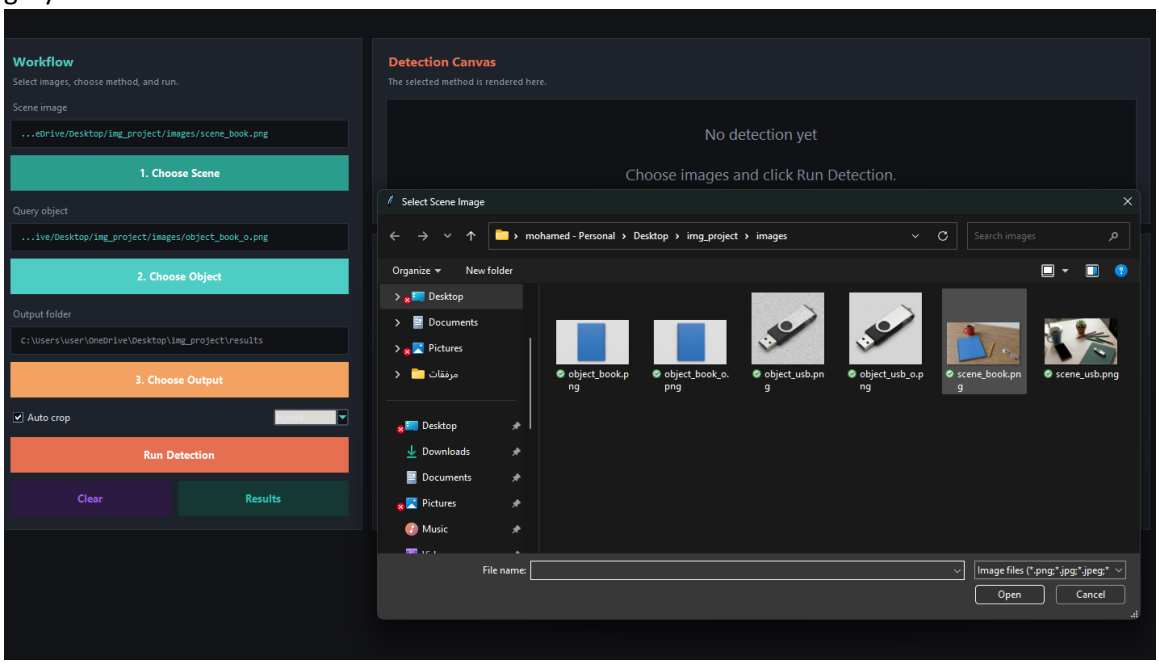
3.Workflow Panel

This panel guides the user step-by-step:

- ### Scene Image Selection

Label: "Scene image"

Button: "1. Choose Scene"Function: Opens a file dialog to load the large scene image. Supported formats: PNG, JPG, JPEG, BMP, WEBP. Visual feedback: The selected file path is displayed in a grey box.

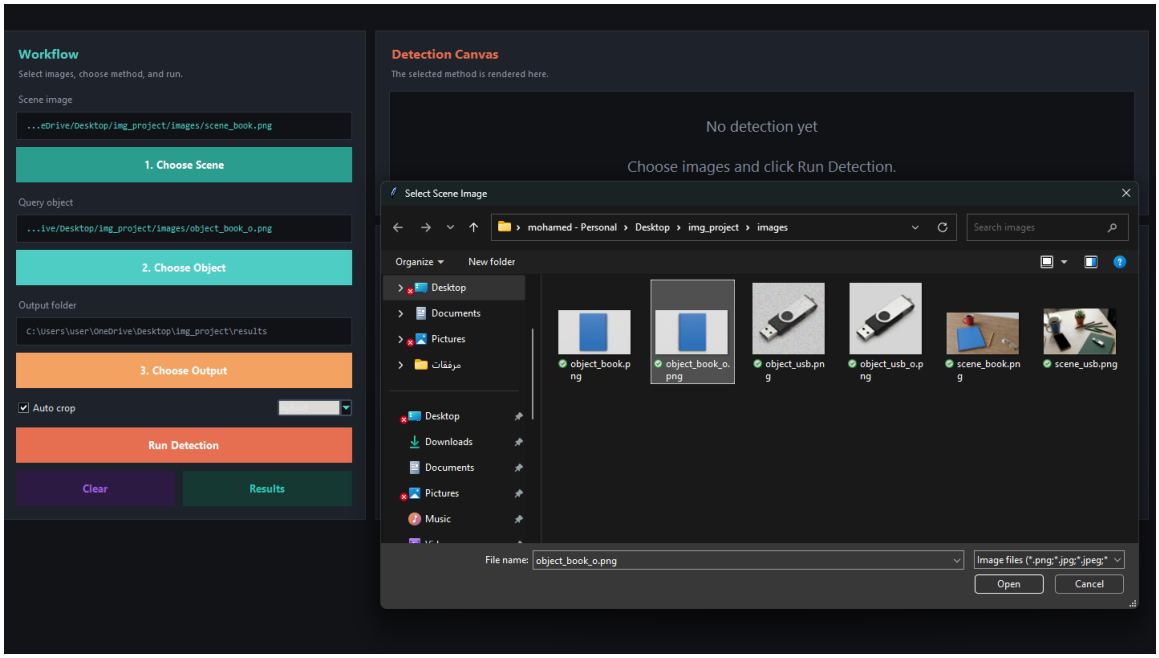


- ## Query Object Selection

Label: "Query object"

Button: "2. Choose Object"

Function: Loads the small object image to be found inside the scene.



- ## Output Folder Selection

Label: "Output folder"

Button: "3. Choose Output"

Function: Allows the user to choose where to save result images and JSON summary files. Defaults to ./results/.

- ## Controls

Checkbox: "Auto crop" – When enabled, the object image is automatically cropped to remove empty borders before processing.

Combobox: Detection method – Choose from template, edge, histogram, or hybrid. The selected method is applied immediately when you click a row in the table.

- ## Action Buttons

"Run Detection" – Starts the detection pipeline. All four methods are executed, results are stored, and the best method is highlighted.

"Clear" – Resets all selections, clears the canvas and table.

"Results" – Opens the output folder in the operating system's file explorer.

4.Preview Panel

- ## Detection Canvas

Title: "Detection Canvas"

Subtitle: "The selected method is rendered here."

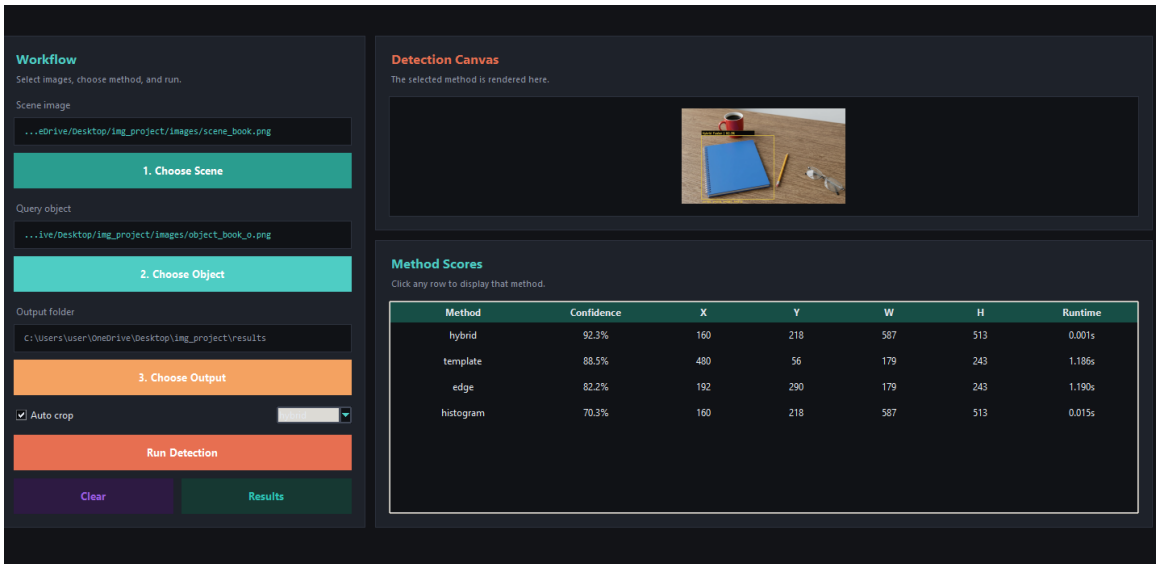
Content: Shows the scene image with a colored bounding box drawn around the detected object. The box color changes per method (e.g., red for hybrid, green for edge). If no detection has been performed, a placeholder text appears.

- ## Method Scores Table

- Title: "Method Scores"

- Columns:

- Method – short name (template, edge, histogram, hybrid)
- Confidence – percentage (0–100%)
- X, Y – top-left coordinates of the bounding box
- W, H – width and height of the bounding box
- Runtime – time taken in seconds
- Interaction: Clicking any row instantly switches the detection canvas to that method and updates the confidence display.



- ## Detection Workflow (Internal Process)

When the user clicks "Run Detection", the following steps occur:

Validate inputs – Check that scene and object paths exist.

- Create output folder if it does not exist.
- Generate timestamp for unique filenames.
- Call detect_all() from the backend module. This function:
- Optionally auto-crops the object.
- Runs all four detection methods.
- Returns a list of DetectionResult objects (each contains method name, confidence score, bounding box, elapsed milliseconds, and extra info).
- Populate the table with the results, sorted by confidence (highest first).

- Save the output image (hybrid result, but can be changed by the user) and a JSON summary file.

- Display the best method on the canvas.

- Update the status bar and stage indicator to "Completed".

5.Custom Widgets

• PillButton

- Inherits from tk.Button.

- Flat design, rounded appearance, custom colors.

- Changes color when the mouse hovers over it.

• Card

- A helper method that creates a framed container with an optional title and subtitle.

- Used for all major panels (Workflow, Detection Canvas, Method Scores).

• Path Label

- A read-only label that displays the selected file/folder path.

- Uses a monospace font and a light grey background.